

'50% Implementation' Means At minimum, by Phase 1 you should have:

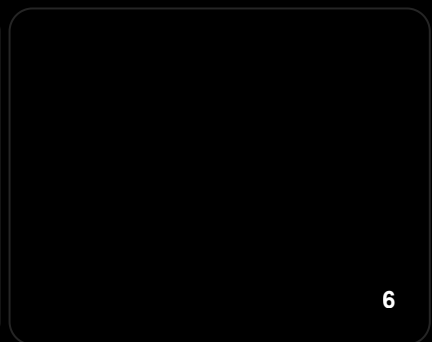
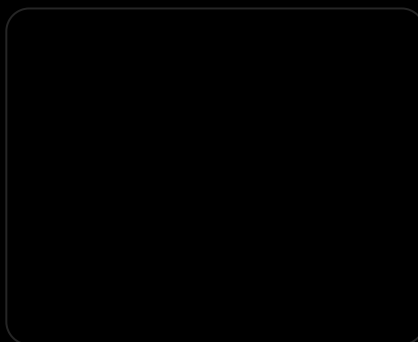
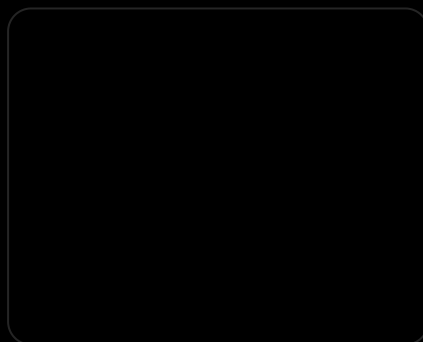
- A finalized methodology and system/architecture design.
- The dataset collected, cleaned, and pre-processed (or a substantial portion of it).
- The core model or pipeline implemented and running end-to-end on at least a subset of data.
- Some preliminary results — even partial metrics, sample outputs, or baseline performance.
- A clear, realistic plan for completing the remaining 50% in Phase 2

Show more

Absolutely. For your evaluation, you should be able to explain **the entire VisionToll pipeline from image → preprocessing → model → bounding boxes/classes → loss → evaluation → comparison → research gap**, and also defend why you selected each model.

I'll explain it as if you are preparing for a **project viva**, including likely questions and answers.

VisionToll — Complete Project Explanation

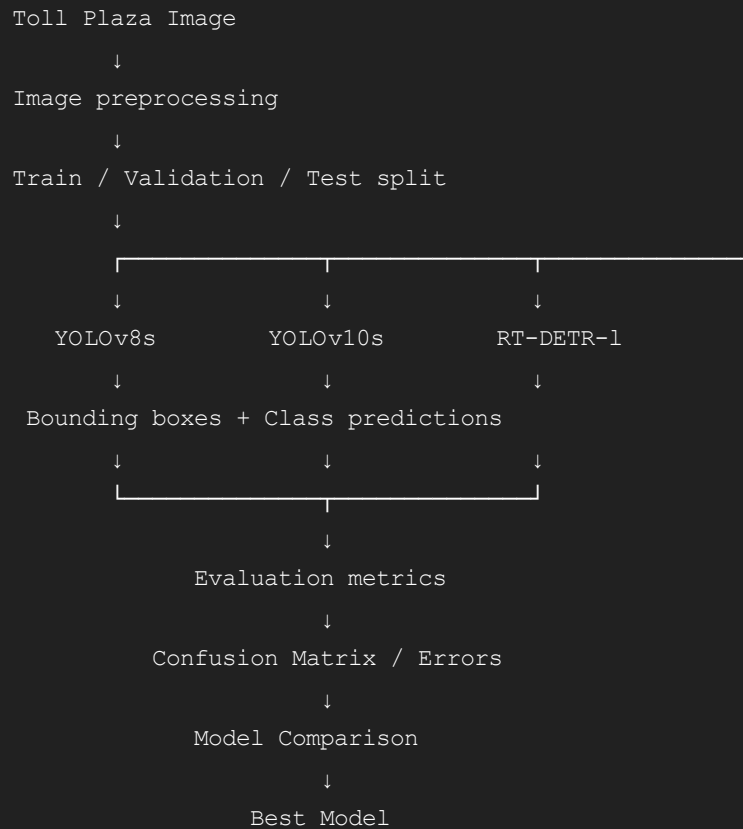


1. First: What exactly is your project?

One-line answer for viva

VisionToll is a deep-learning-based vehicle detection and classification system designed for toll-plaza environments, where we compare CNN-based YOLOv8, NMS-free YOLOv10, and transformer-based RT-DETR to determine which architecture is better at identifying visually similar and difficult vehicle categories while maintaining real-time performance.

Your system essentially does this:



2. What problem are you solving?

Traditional toll plazas need to identify vehicles such as:

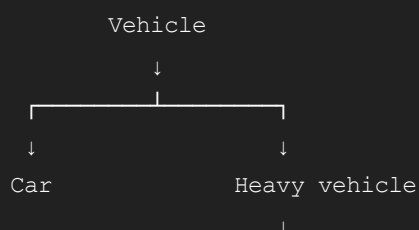
- Cars
- Buses
- Trucks
- Pickups
- Motorcycles

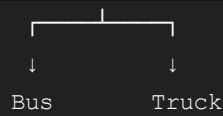
The difficulty isn't simply detecting **"there is a vehicle."**

The difficult part is:

Which type of vehicle is it?

For example:





A toll-plaza camera can have:

- Different viewing angles
- Partial occlusion
- Small/distant vehicles
- Different lighting
- Vehicles overlapping each other
- Similar-looking vehicle categories

Therefore your research question is essentially:

Can a transformer-based detector such as RT-DETR handle difficult vehicle classification better than conventional real-time CNN detectors such as YOLOv8 and YOLOv10?

3. Why object detection instead of classification?

This is a **very important viva question**.

Image classification

Classification answers:

"What is present in this image?"

For example:

Image → Car

But suppose your image contains:

Car Bus Truck

Classification doesn't tell you where each vehicle is.

Object detection

Object detection gives:

Image

↓

[]

Car

Bus

Truck

So the model predicts:

1. **Bounding box**
2. **Class**
3. **Confidence score**

For example:

```
Car
x1 = 120
y1 = 80
x2 = 300
y2 = 250
confidence = 0.94
```

Therefore VisionToll is an **object detection** problem.

4. What dataset are you using?

You are using the **Indian Vehicle Dataset from Kaggle**.

Your dataset contains:

```
Images
+
YOLO annotation files
```

The annotations tell the model:

"There is a vehicle of this class at this particular location."

A YOLO label generally looks like:

```
class_id x_center y_center width height
```

For example:

```
0 0.52 0.47 0.30 0.25
```

These coordinates are normally **normalized between 0 and 1**.

So:

```
x_center = 0.52
y_center = 0.47
width    = 0.30
height   = 0.25
```

5. What is `data.yaml`?

This is another likely viva question.

`data.yaml` tells Ultralytics where your data is and what the classes are.

Conceptually:

YAML

```
path: /dataset

train: train/images
val:   valid/images
test:  test/images

names:
  0: car
  1: bus
  2: pickup
  3: motorcycle
  4: truck
```

The exact class numbering should be whatever your actual dataset uses.

Why is this important?

The model needs to know:

- Where training images are
- Where validation images are
- Where test images are
- How many classes exist
- What each class ID means

6. Train / Validation / Test

This is fundamental.

Training set

Used to learn the model parameters.

```
Training data
  ↓
Model
  ↓
Loss
  ↓
Backpropagation
  ↓
Weight update
```

Validation set

Used during development to determine:

"Is the model improving on data it hasn't trained directly on?"

Test set

Used for final evaluation.

The test set should ideally remain unseen during training.

So:

```
TRAIN
  ↓
Learn

VALIDATION
  ↓
Tune / monitor

TEST
  ↓
Final performance
```

7. What does preprocessing mean?

Before training, images have to be prepared.

Typical operations include:

Resizing

Images can have different dimensions.

They are resized to the model's training image size.

For example:

```
1920 × 1080
    ↓
640 × 640
```

Normalization

Pixel values are represented in a suitable numerical range.

Data augmentation

Artificially creates variation in training images.

Examples:

- Horizontal flipping
- Scaling
- Cropping
- Mosaic
- MixUp

The objective is:

Make the model more robust to variations it may encounter in real-world toll-plaza images.

8. What is augmentation?

Suppose you have:

```
Original image
```

You can generate modified versions:

Original

↓

Flip

↓

Scale

↓

Crop

↓

Mosaic

↓

MixUp

The model therefore doesn't simply memorize the training images.

It learns more general features.

9. What is the Ultralytics framework?

Your project uses the `ultralytics` Python framework.

The important idea is:

Instead of implementing three completely different training pipelines ourselves, we use a common framework for loading data, training, validation and evaluation.

This helps make the experiment more consistent.

Your experimental pipeline is therefore:

Same dataset

↓

Same train/val/test split

↓

Same general training environment

↓

Same evaluation methodology

↓

Different model architecture

↓

Compare results

That makes the comparison more meaningful.

10. Model 1 — YOLOv8s

Now we get into the most important part.

YOLOv8s

YOLO means:

You Only Look Once

YOLO is a **single-stage object detector**.

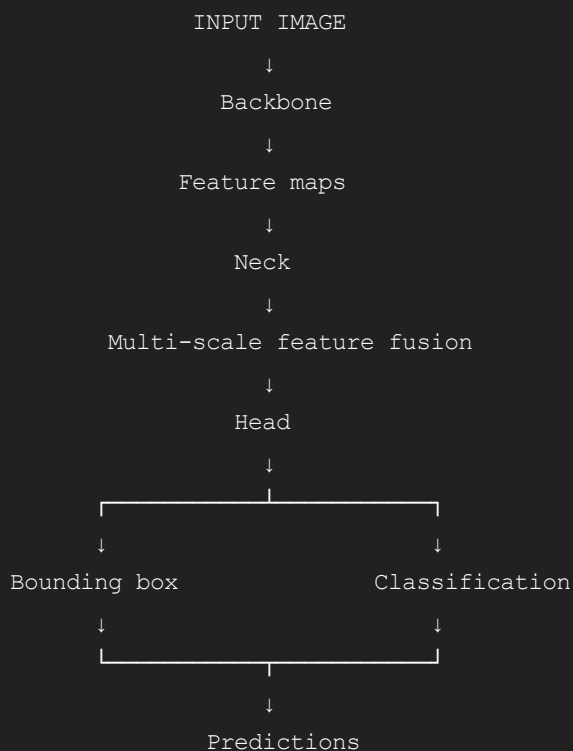
The basic idea is:

```
Image
  ↓
Neural network
  ↓
Features
  ↓
Bounding boxes + classes + confidence
```

Unlike older approaches that first generate region proposals and then classify them, YOLO performs detection in essentially one integrated pipeline.

11. YOLOv8 architecture

At a simplified level:



Three important components:

1. Backbone

Extracts visual features.

For example:

```
Edges
↓
Shapes
↓
Vehicle parts
↓
High-level vehicle features
```

2. Neck

Combines information at different scales.

Why?

Because:

```
Large nearby vehicle
```

and

```
Small distant vehicle
```

require different feature resolutions.

3. Detection head

Produces:

- Bounding box
- Class
- Confidence

12. What does CNN actually do?

CNN = **C**onvolutional **N**eural **N**etwork.

A convolution filter moves across the image.

Initially it can learn:

Edges
Lines
Corners

Then deeper layers learn:

Wheels
Windows
Headlights
Vehicle shapes

Then even deeper layers learn:

Car-like structure
Truck-like structure
Bus-like structure

So:

Pixels
↓
Edges
↓
Shapes
↓
Parts
↓
Objects

This hierarchical feature extraction is one of the major strengths of CNNs.

13. What is CSP/C2f-style feature extraction?

Your description mentions CSPDarknet.

The important viva-level idea is:

Cross Stage Partial-style processing allows the network to reuse and combine features efficiently while controlling computational cost.

In YOLOv8 terminology, the architecture uses **C2f modules** extensively.

You don't need to memorize every convolution layer unless your evaluator specifically asks.

14. Why YOLOv8s?

The **s** means **small**.

YOLO models come in different sizes.

Conceptually:

```
YOLOv8n → nano  
YOLOv8s → small  
YOLOv8m → medium  
YOLOv8l → large  
YOLOv8x → extra large
```

Larger models generally have:

- More parameters
- More computation
- Potentially better accuracy
- Higher inference cost

You selected **YOLOv8s** because you're interested in real-time detection.

15. YOLOv8 result

Your reported result:

```
mAP@50      = 0.5847  
mAP@50-95   = 0.3891  
Parameters  ≈ 11.1M  
Latency     ≈ 12.4 ms
```

This establishes your **CNN baseline**.

16. Model 2 — YOLOv10s

Now the second experiment.

YOLOv10 is also a YOLO-family detector, but one of its major goals is reducing unnecessary post-processing.

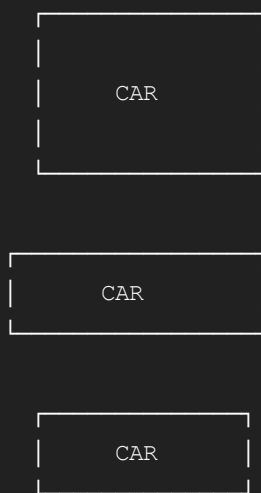
The important concept is:

NMS

NMS = **Non-Maximum Suppression**

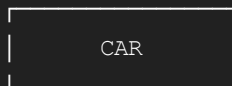
17. Why do we need NMS?

Suppose the detector predicts:



The model may generate multiple boxes for the same vehicle.

We only want:



NMS removes redundant boxes.

18. How NMS works

Suppose:

```
Box A → confidence 0.92  
Box B → confidence 0.81  
Box C → confidence 0.72
```

If they overlap heavily:

1. Keep highest-confidence box.
2. Calculate overlap with other boxes.
3. Remove boxes whose overlap is above a threshold.
4. Continue.

The overlap is measured using:

IoU

IoU = **Intersection over Union**

$$\text{IoU} = \frac{\text{Area of intersection}}{\text{Area of union}}$$

If two boxes overlap perfectly:

$$\text{IoU} = 1$$

If they don't overlap:

$$\text{IoU} = 0$$

19. Why YOLOv10 is interesting

YOLOv10 introduces an **NMS-free end-to-end detection design**, based on consistent dual assignments.

The basic concept is:

```
Training
├─ one-to-many assignment
└─ one-to-one assignment

Inference
  ↓
one-to-one predictions
  ↓
less dependence on NMS
```

Therefore, the model aims to reduce:

- Post-processing
 - Latency
 - Computational overhead
-

20. What is the important YOLOv10 experiment?

Your research question wasn't:

"Is YOLOv10 always better than YOLOv8?"

It was more interesting:

Does removing NMS and optimizing latency provide a useful detector for toll-plaza vehicle detection?

Your result:

```
YOLOv8s  
mAP@50 = 0.5847  
  
YOLOv10s  
mAP@50 = 0.5508
```

Therefore:

```
YOLOv8s > YOLOv10s
```

in your particular experiment.

21. Important viva point

Do **NOT** say:

"YOLOv10 is worse than YOLOv8."

Say:

"YOLOv10 achieved lower accuracy on our particular dataset and experimental configuration. This does not imply that YOLOv10 is generally inferior; it indicates that under our dataset, training configuration and class distribution, YOLOv8s performed better."

That is a much more scientifically correct answer.

22. Model 3 — RT-DETR

This is the most important model for your research contribution.

RT-DETR:

Real-Time Detection Transformer

It is fundamentally different from the YOLO CNN-based approach.

Instead of relying primarily on CNN feature processing plus conventional detection mechanisms, RT-DETR uses a **Transformer-based detection architecture**.

23. Why Transformers?

CNNs are very good at extracting local patterns.

For example:

```
Wheel
Window
Headlight
Edge
Corner
```

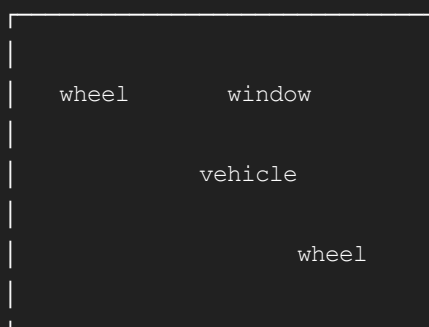
But understanding the relationship between distant regions can be more challenging.

Transformers use:

Self-attention

Self-attention allows the model to consider relationships between different parts of the image.

Simplified example:



The model can reason about relationships among these regions.

24. What is self-attention?

This is a common viva question.

The intuition:

Self-attention allows one feature to determine how important other features are when interpreting it.

Suppose a feature represents:

```
vehicle roof
```

The network can determine that features corresponding to:

```
wheels  
windows  
body  
front/rear structure
```

are relevant to interpreting the object.

Mathematically, attention is commonly represented as:

$$Attention(Q, K, V) = softmax \left(\frac{QK^T}{\sqrt{d}} \right) V$$

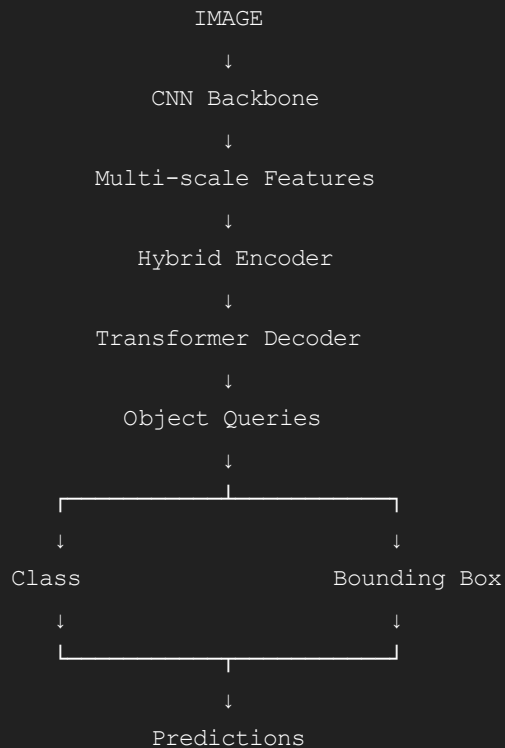
Where:

- Q = Query
- K = Key
- V = Value
- d = feature dimension

You don't need to derive it unless asked.

25. RT-DETR architecture

Simplified:



26. What is the backbone?

RT-DETR still needs a CNN-style backbone to extract initial visual features.

Depending on the implementation/configuration, different backbones can be used.

Its job is:

```
Image
↓
Low-level features
↓
Mid-level features
↓
High-level features
```

27. What is the hybrid encoder?

This is an important RT-DETR component.

The backbone produces multi-scale feature maps.

The encoder processes them efficiently so that the transformer doesn't have to perform expensive global processing on every raw pixel.

Conceptually:

```
CNN feature maps
      ↓
Efficient feature interaction
      ↓
Transformer-ready representations
```

28. What is the Transformer decoder?

This is where object queries are used.

The decoder essentially asks:

"What objects exist in this image, and where are they?"

It produces object predictions.

For example:

```
Query 1 → Car
Query 2 → Truck
Query 3 → Bus
Query 4 → Motorcycle
```

Each query can correspond to a predicted object.

29. Why RT-DETR might help your problem

Your research gap involves:

Visual similarity

For example:

```
Truck ↔ Bus
```

They can look similar from a toll-camera viewpoint.

A purely local feature may not be sufficient.

Self-attention can potentially combine information from different parts of the image.

Therefore your hypothesis was:

Global contextual reasoning through attention may improve discrimination between visually similar vehicle classes.

And your experimental result supports that hypothesis.

But be careful with the word "proves."

Say:

"Our results support the hypothesis."

rather than:

"This definitively proves that transformers are always better."

30. Your most important results

Your reported table is:

Model	Type	mAP@50	mAP@50-95	Parameters	Latency
YOLOv8s	CNN	0.5847	0.3891	11.1M	12.4 ms
YOLOv10s	NMS-free CNN	0.5508	0.3344	8.1M	12.3 ms
RT-DETR-L	Transformer	0.6786	0.4535	32.8M	40.3 ms

The first thing the evaluator will notice:

```
RT-DETR
  ↑
Highest accuracy
```

but:

```
RT-DETR
  ↑
Highest latency
```

Therefore your result isn't simply:

"RT-DETR is best."

It is:

RT-DETR gives the highest detection accuracy, while YOLOv8s and YOLOv10s provide significantly lower latency.

That's a much stronger scientific conclusion.

31. What does mAP mean?

Very important.

mAP = mean Average Precision

Object detection has two important concepts:

Precision

Of the objects predicted:

How many were correct?

$$Precision = \frac{TP}{TP + FP}$$

Recall

Of the objects that actually exist:

How many did we detect?

$$Recall = \frac{TP}{TP + FN}$$

Where:

- TP = True Positive
 - FP = False Positive
 - FN = False Negative
-

32. What is AP?

AP = Average Precision

It summarizes the precision-recall relationship for a particular class.

For example:

```
AP(car)
AP(bus)
AP(truck)
AP(pickup)
AP(motorcycle)
```

Then:

$$mAP = \frac{\sum AP_{classes}}{N}$$

So mAP averages the AP across classes.

33. What is mAP@50?

mAP@50 uses:

IoU threshold = 0.50

A predicted bounding box is considered sufficiently overlapping with the ground-truth box when IoU reaches the specified threshold.

Your:

```
RT-DETR mAP@50 = 0.6786
```

means the mean AP across your classes at IoU 0.50 is approximately **67.86%**.

34. What is mAP@50-95?

This is stricter.

It averages performance over multiple IoU thresholds:

```
0.50
0.55
0.60
0.65
...
0.95
```

Therefore mAP@50-95 evaluates localization more rigorously.

Your results:

```
YOLOv8s → 0.3891
YOLOv10s → 0.3344
RT-DETR → 0.4535
```

Again:

```
RT-DETR > YOLOv8s > YOLOv10s
```

35. Why is mAP@50-95 lower?

Because stricter localization requirements make the task harder.

At:

```
IoU = 0.50
```

a box can be somewhat imperfect and still count as sufficiently overlapping.

At:

```
IoU = 0.95
```

the predicted box must be extremely close to the ground truth.

Therefore:

```
mAP@50 > mAP@50-95
```

is completely normal.

36. Your class-wise RT-DETR results

You have:

```
Car      0.850
Bus      0.495
Pickup   0.703
Motorcycle 0.680
Truck    0.665
```

Notice something important:

```
Car
↑
0.850
```

while:

```
Bus
↓
0.495
```

This tells you that the overall mAP hides important class-level behavior.

37. Why is Car performing so well?

According to your analysis, your dataset has many more car instances.

You reported:

```
Cars      = 398
Buses     = 6
Trucks    = 8
```

This is a **severe class imbalance**.

Think of it like studying:

```
398 car examples
6 bus examples
8 truck examples
```

A model gets vastly more opportunity to learn what cars look like.

Therefore high car performance does not necessarily mean the system is equally good at every vehicle category.

38. Very important correction for your viva

Don't say:

"The model is overfitting because there are more cars."

Class imbalance and overfitting are **not the same thing**.

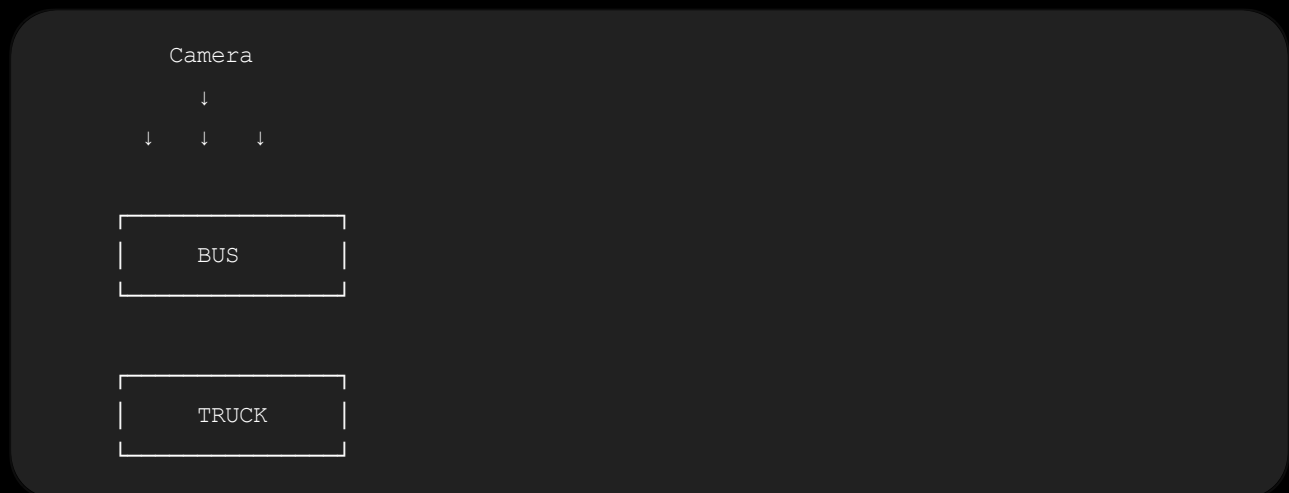
Better:

"The severe class imbalance can bias the model toward the majority class and makes learning minority classes more difficult. Additional analysis would be required to establish overfitting specifically."

This distinction can impress your evaluator.

39. Why Bus and Truck are difficult?

Imagine a toll camera:



Both can have:

- Large rectangular bodies
- Similar proportions
- Similar colors
- Similar viewing angles
- Partial occlusion

Therefore:



This directly relates to your research gap.

40. YOLOv10 class results

You reported:

Car	0.837
Bus	0.211
Pickup	0.734
Motorcycle	0.628
Truck	0.344

The important observation is:

Car → strong

Bus → very weak

Truck → weak

This supports the conclusion that heavy vehicle categories are particularly challenging.

41. Confusion matrix

A confusion matrix answers:

Which classes are being confused with which other classes?

Conceptually:

Actual \ Predicted	Car	Bus	Pickup	Motorcycle	Truck
Car	✓				
Bus		✓			← confusion
Pickup			✓		
Motorcycle				✓	
Truck		← confusion			✓

The diagonal represents correct predictions.

Off-diagonal cells represent errors.

42. What is error analysis?

Instead of simply saying:

"RT-DETR got 0.6786."

you inspect:

- Which classes fail?
- Which images fail?
- Why do they fail?
- Are objects too small?
- Are vehicles occluded?
- Are bus/truck shapes similar?
- Are annotations incorrect?
- Is there class imbalance?

This makes your project much more research-oriented.

43. Your research gap

Your literature review identified:

Gap 1 — Real-time processing

A toll system cannot take several seconds to process one frame.

It needs fast inference.

Gap 2 — Visually similar vehicles

For example:

Truck vs Bus
SUV vs Car

Gap 3 — Difficult visual conditions

Including:

- Occlusion
- Scale variation
- Camera perspective
- Small vehicles

Your experiment addresses these by comparing:

CNN
↓

YOLOv8

NMS-free CNN

↓

YOLOv10

Transformer

↓

RT-DETR

44. Why compare three models?

This is an excellent viva question.

You can answer:

"We intentionally selected architectures representing three different design philosophies. YOLOv8s serves as a conventional real-time CNN baseline, YOLOv10s tests whether an NMS-free design can improve efficiency, and RT-DETR represents transformer-based global contextual reasoning. This gives us a meaningful architectural comparison rather than comparing three minor variations of the same model."

That's a strong answer.

45. Why not compare YOLOv8 and YOLOv9?

You could, but that wouldn't address your research question as strongly.

Your comparison is:

CNN baseline

vs

NMS-free CNN

vs

Transformer

Therefore you're comparing **architectural approaches**, not just model versions.

46. Why Kaggle?

You selected Kaggle because:

- Dataset is available there.
- GPU environments are available.
- T4/P100 GPUs can accelerate training.
- The environment is reproducible.
- It avoids depending entirely on your local computer's GPU.

Your workflow:

```
Kaggle
  ↓
Dataset
  ↓
Notebook
  ↓
GPU
  ↓
Training
  ↓
Evaluation
```

47. What is an epoch?

You trained for:

```
50 epochs
```

One epoch means:

| The model has processed the entire training dataset once.

So:

```
1 epoch → entire training set once
50 epochs → entire training set approximately 50 times
```

Not necessarily 50 weight updates—there are normally many **batches** per epoch.

48. What is batch size?

Suppose:

```
Batch size = 16
```

The model processes:

```
16 images
↓
calculate predictions
↓
calculate loss
↓
backpropagation
↓
update weights
```

Then another batch.

This continues until the entire training dataset has been processed.

49. What is loss?

Loss tells the model:

How wrong are my predictions?

For object detection, there are multiple error components.

Conceptually:

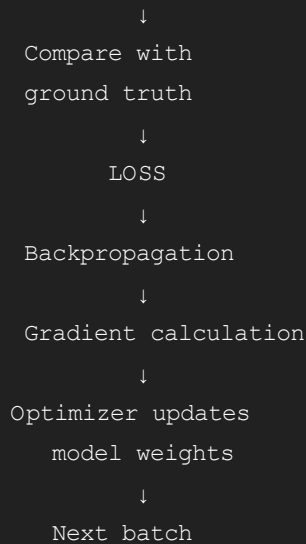
```
Total Loss
├──
│   ├── Box/localization loss
│   ├── Classification loss
│   └── Other detection-specific losses
```

The optimizer attempts to minimize the loss.

50. Training process

This is one of the most important things you should be able to explain.

```
IMAGE
↓
MODEL
↓
Predictions
```



Repeated many times:

batch → loss → gradient → update

until training progresses through many epochs.

51. What is backpropagation?

Backpropagation determines:

How much each model parameter contributed to the error.

It calculates gradients using the chain rule.

Then the optimizer changes the weights in a direction that reduces the loss.

Simplified:

$$w_{new} = w_{old} - \eta \nabla L$$

where:

- w = model weight
- η = learning rate
- L = loss

52. What is an optimizer?

The optimizer updates the model's parameters.

You can think:

```
Model makes mistake
  ↓
Loss calculated
  ↓
Optimizer decides
how weights should change
  ↓
Model improves
```

The exact optimizer and hyperparameters depend on the training configuration.

53. What is inference?

Training:

```
Image + ground truth
  ↓
learn
```

Inference:

```
New image
  ↓
Model
  ↓
Prediction
```

For example:

```
Car      0.96
Truck    0.82
Motorcycle 0.91
```

along with bounding boxes.

No weight updates happen during normal inference.

54. What is latency?

Your latency results:

```
YOLOv8s  = 12.4 ms
YOLOv10s = 12.3 ms
```

Latency means:

How long it takes to process an image/frame under the stated measurement conditions.

Lower is better.

55. Convert latency to FPS

Approximately:

$$FPS = \frac{1000}{latency(ms)}$$

For RT-DETR:

$$FPS = \frac{1000}{40.3}$$

≈ 24.8 FPS

So your statement of approximately:

25 FPS

is reasonable.

YOLOv8:

$$1000/12.4 \approx 80.6FPS$$

YOLOv10:

$$1000/12.3 \approx 81.3FPS$$

So your models are approximately:

YOLOv8s → 81 FPS

YOLOv10s → 81 FPS

RT-DETR → 25 FPS

Important: FPS calculated from latency assumes the latency measurement is directly comparable and excludes other pipeline overhead such as image capture, preprocessing, postprocessing, and I/O.

56. Is 25 FPS real-time?

Generally, yes, **25 FPS is within the commonly used real-time video range.**

But don't say:

"RT-DETR is definitely suitable for every toll system."

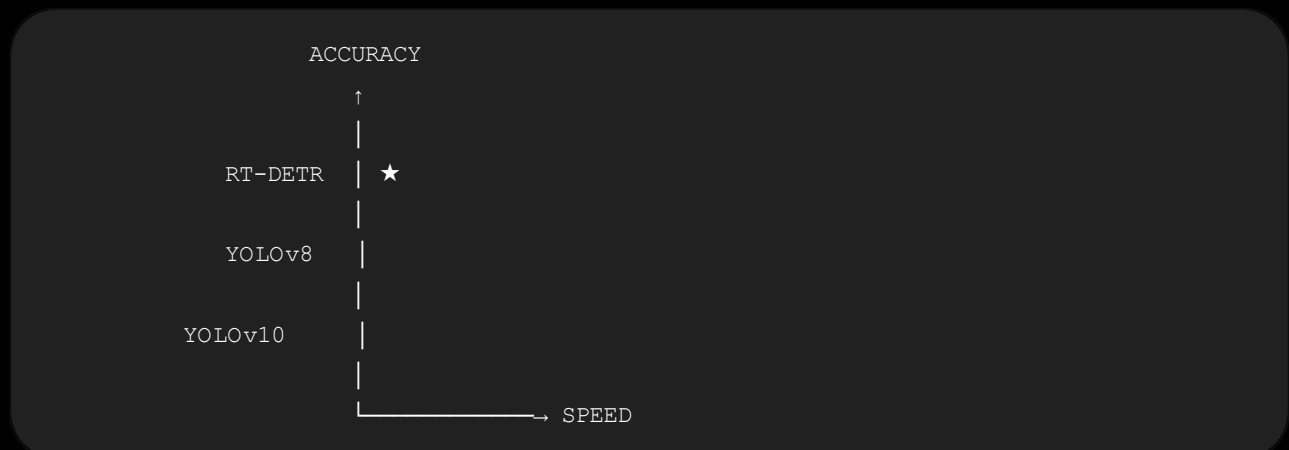
Instead:

"Under our Kaggle T4 measurement conditions, RT-DETR achieved approximately 25 FPS, which is compatible with many real-time video applications. Deployment suitability would still depend on the target hardware and complete end-to-end pipeline."

Excellent viva answer.

57. The most interesting trade-off

Your results actually create a very nice conclusion:



RT-DETR:

Best accuracy

but

Highest latency

YOLOv8/YOLOv10:

Much faster

but

Lower accuracy

Therefore your system demonstrates an:

Accuracy–Latency Trade-off

58. Is RT-DETR actually the "winner"?

You need to answer carefully.

Accuracy winner

Yes:

```
RT-DETR  
mAP50 = 0.6786
```

Speed winner

YOLOv10:

```
12.3 ms
```

slightly beats YOLOv8:

```
12.4 ms
```

Overall

If the priority is:

highest detection accuracy

RT-DETR is your winner.

If the priority is:

minimum latency

YOLOv10s is your winner.

This nuance is important.

59. Why does RT-DETR have more parameters?

You reported:

```
YOLOv8s    ≈ 11.1M
YOLOv10s   ≈ 8.1M
RT-DETR-l  ≈ 32.8M
```

More parameters generally mean a larger model capacity.

RT-DETR-l is substantially larger than your YOLO models.

Therefore:

```
More model capacity
      +
Transformer architecture
      ↓
Potentially stronger representation
      ↓
Higher accuracy
```

But:

```
More computation
      ↓
Higher latency
```

This contributes to the trade-off you observed.

60. VERY IMPORTANT: RT-DETR-l vs YOLOv8s is not perfectly size-matched

This is something your evaluator may catch.

You're comparing:

```
YOLOv8s
YOLOv10s
RT-DETR-l
```

The first two are **small** models, while RT-DETR-l is a **large** model.

Therefore you should NOT claim:

"Transformer architecture alone caused the accuracy improvement."

Because architecture size/capacity is also a confounding factor.

A scientifically safer conclusion is:

"Under our selected configurations, RT-DETR-L achieved the highest accuracy. The result is consistent with our hypothesis that transformer-based contextual reasoning can benefit visually challenging detection, although the larger model capacity is also a factor and should be controlled in future experiments."

This is an excellent Phase 2 direction.

61. Another important issue: class imbalance

Your dataset has approximately:

Car	398
Bus	6
Truck	8

This is extremely imbalanced.

Imagine training a model where:

98% → Car
2% → Other

The model sees very few minority-class examples.

Therefore your Phase 2 should consider:

More minority-class data

Especially:

Bus
Truck

Augmentation

Such as:

Mosaic
MixUp
CutMix

where appropriate.

Class-aware sampling

Try to expose the model to minority classes more frequently.

Loss modifications

Potentially investigate class-sensitive or focal-style approaches, depending on the model/framework.

62. Why shouldn't you simply duplicate bus images?

Because the model can memorize them.

For example:

```
Bus image 1  
Bus image 1  
Bus image 1  
Bus image 1
```

doesn't provide four independent examples.

Better:

```
Bus image 1 → crop/scale/augmentation  
Bus image 2 → augmentation  
Bus image 3 → augmentation  
...
```

But ultimately, **more real diverse bus/truck images** are preferable.

63. Your Phase 1 status

According to your handoff:

Completed

- ✓ Dataset downloaded
- ✓ Dataset verified
- ✓ Train/validation/test split
- ✓ `data.yaml`
- ✓ Kaggle GPU environment

✓ Ultralytics installed

✓ YOLOv8s trained

✓ YOLOv10s trained

✓ RT-DETR trained

✓ 50 epochs

✓ Evaluation

✓ Confusion matrices

✓ Preliminary metrics

This satisfies the core idea of:

50% Implementation

You have:

Methodology	✓
Dataset	✓
Preprocessing	✓
Core pipeline	✓
Models	✓
Training	✓
Evaluation	✓
Preliminary results	✓
Phase 2 plan	🔄

64. What remains for Phase 2?

Your next stage should be more than simply:

"Train again."

You should improve the scientific depth.

Phase 2

```
Class imbalance handling
    ↓
Improved augmentation
    ↓
Hyperparameter tuning
```

↓
Better minority-class performance
↓
Additional experiments
↓
Failure-case analysis
↓
Robustness testing
↓
Final model
↓
Deployment/demo

65. What should your final system eventually do?

The final VisionToll application could look like:

```
graph TD
    CAMERA --> Video_Image[Video / Image]
    Video_Image --> Vehicle_Detection[Vehicle Detection]
    Vehicle_Detection --> Car
    Vehicle_Detection --> Bus
    Vehicle_Detection --> Truck
    Car --> Vehicle_classification[Vehicle classification]
    Bus --> Vehicle_classification
    Truck --> Vehicle_classification
    Vehicle_classification --> Toll_classification[Toll classification]
    Toll_classification --> Optional_integration[Optional integration]
```

Potentially later:

```
graph TD
    Vehicle_detected[Vehicle detected] --> Vehicle_class[Vehicle class]
    Vehicle_class --> Lane_tracking[Lane tracking]
    Lane_tracking --> Toll_category[Toll category]
    Toll_category --> Billing_transaction[Billing / transaction system]
```

But don't claim you've implemented those parts if you haven't.

66. Difference between detection and toll classification

Your current project is primarily:

Vehicle detection and classification

It is not necessarily a complete toll-payment system.

For example:

```
RT-DETR
↓
"Truck"
```

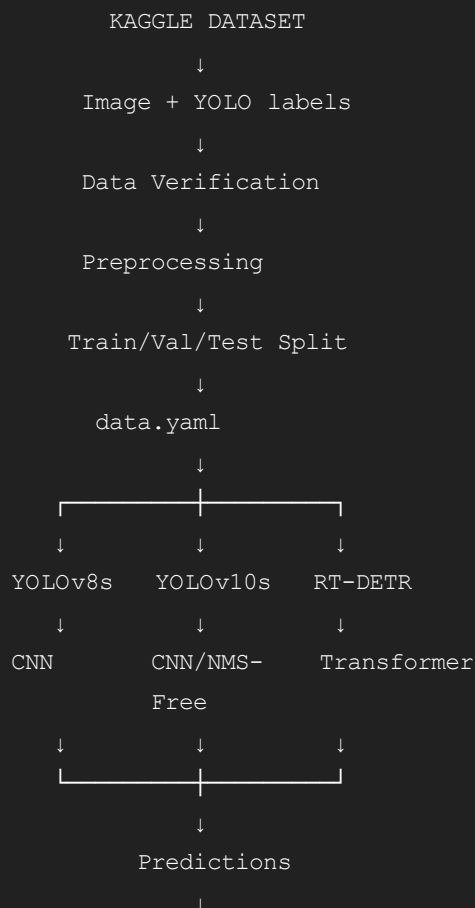
doesn't automatically mean:

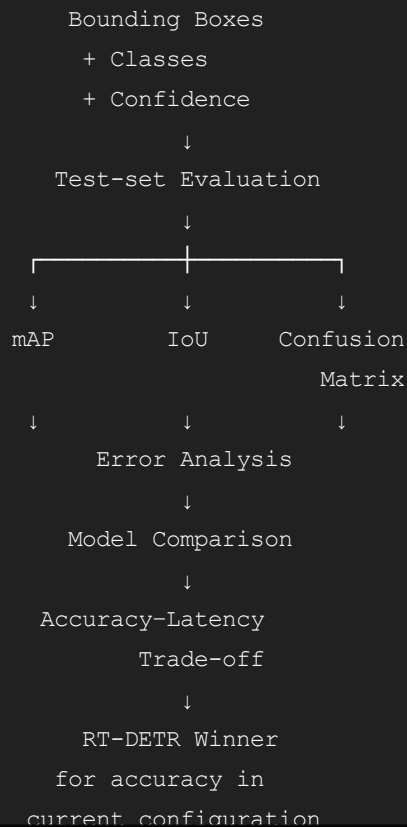
```
₹XXX toll
```

The toll amount would require a separate business/rules layer.

67. Your complete technical architecture

You should memorize this:





68. Your 30-second project explanation

If the evaluator says:

"Explain your project."

Say:

"Our project, VisionToll, is an object detection system for vehicle classification in toll-plaza environments. The main challenge is distinguishing visually similar vehicle categories under conditions such as occlusion, scale variation and different viewpoints. We use an Indian Vehicle Dataset from Kaggle and convert it into a train, validation and test pipeline using YOLO-format annotations. We compare three architectures under our experimental setup: YOLOv8s as the CNN baseline, YOLOv10s as an NMS-free low-latency detector, and RT-DETR-I as a transformer-based detector. We trained the models for 50 epochs and evaluated them using $mAP@50$, $mAP@50-95$, confusion matrices and inference latency. RT-DETR achieved the highest test $mAP@50$ of 0.6786, compared with 0.5847 for YOLOv8s and 0.5508 for YOLOv10s. However, RT-DETR had higher latency at about 40.3 milliseconds, compared with approximately 12 milliseconds for the YOLO models. Our Phase 1 therefore demonstrates an accuracy-latency trade-off and supports our hypothesis that transformer-based contextual reasoning can be beneficial for visually challenging vehicle detection. Phase 2 will focus on class imbalance, minority-class performance, hyperparameter tuning and robustness."

That's a very strong answer.

69. Questions they are VERY likely to ask

Q1. Why did you choose YOLOv8?

Answer:

"YOLOv8 provides a strong and widely used real-time CNN baseline. It gives us a reference point against which newer architectures can be compared."

Q2. Why YOLOv10?

"YOLOv10 is designed around end-to-end NMS-free detection and aims to reduce inference and post-processing latency. We wanted to investigate whether this efficiency advantage would be useful in toll-plaza vehicle detection."

Q3. Why RT-DETR?

"RT-DETR introduces transformer-based attention and global contextual reasoning. Since our research gap includes visually similar vehicle categories and difficult scenes, we hypothesized that this architecture could better capture relationships between different visual regions."

Q4. Why object detection?

"Because we need both the vehicle category and its location in the image. Classification alone only provides an image-level label."

Q5. What is IoU?

"Intersection over Union measures the overlap between a predicted bounding box and the ground-truth bounding box."

$$IoU = \frac{Intersection}{Union}$$

Q6. What is mAP?

"Mean Average Precision is the mean of Average Precision across the object classes, evaluated using specified IoU thresholds."

Q7. Difference between mAP@50 and mAP@50-95?

"mAP@50 evaluates at IoU 0.50, while mAP@50-95 averages AP across IoU thresholds from 0.50 to 0.95, making it a stricter evaluation of localization."

Q8. What is NMS?

"Non-Maximum Suppression removes redundant overlapping bounding boxes and keeps the most confident prediction."

Q9. Why is YOLOv10 NMS-free?

"YOLOv10 uses an end-to-end design with one-to-one matching for inference, allowing it to avoid the conventional NMS post-processing step."

Q10. What is a CNN?

"A convolutional neural network learns hierarchical spatial features from images, progressing from low-level patterns such as edges to higher-level features such as object parts and complete objects."

70. RT-DETR viva questions

Q11. What is a Transformer?

"A transformer is a neural network architecture that uses attention mechanisms to model relationships between different elements of the input."

Q12. What is self-attention?

"Self-attention allows each feature to consider other features and assign them different importance when producing its representation."

Q13. Why is attention useful here?

"Vehicles such as buses and trucks can have similar local visual features. Attention can incorporate broader contextual information when forming the object representation."

Q14. Does RT-DETR completely eliminate CNNs?

No.

This is important.

"No. RT-DETR still uses a CNN-based backbone to extract visual features. The transformer components are then used for efficient global reasoning and object decoding."

71. Dataset questions

Q15. Why Kaggle?

"It provides accessible dataset hosting and GPU notebook infrastructure, allowing us to train the models without depending entirely on local hardware."

Q16. Why train/validation/test?

"Training learns the parameters, validation helps monitor and tune the system, and the test set provides an unbiased final evaluation on unseen data."

Q17. Why data augmentation?

"To increase variation in training data and improve generalization to changes in scale, position, orientation and appearance."

72. Results questions

Q18. Which model performed best?

"RT-DETR-L achieved the highest test-set mAP@50 of 0.6786."

Q19. Which was fastest?

"YOLOv10s, at approximately 12.3 milliseconds per image under our reported Kaggle T4 measurement."

Q20. Why did YOLOv10 perform worse?

Don't claim a definite cause.

Say:

"On this dataset and configuration, YOLOv10 achieved lower accuracy. Possible factors include the severe class imbalance, dataset characteristics and the fact that our models were not exhaustively hyperparameter-tuned. We therefore treat this as an empirical result rather than a universal statement about YOLOv10."

Excellent answer.

73. Class imbalance question

Q21. What is your biggest dataset problem?

"Severe class imbalance. The dataset contains substantially more car instances than bus and truck instances."

Q22. How will you fix it?

"In Phase 2 we plan to investigate additional minority-class data, class-aware sampling, stronger augmentation and appropriate loss strategies, followed by controlled evaluation to determine which intervention actually improves minority-class performance."

74. Research questions

Q23. What is your research gap?

Answer:

"Existing approaches can achieve good vehicle detection, but toll-plaza environments introduce challenges involving visually similar vehicle categories, occlusion, scale

variation and real-time constraints. Our study addresses this by conducting a controlled comparison between CNN-based, NMS-free CNN-based and transformer-based detectors."

75. What is your contribution?

This is probably the **most important research question**.

Say:

"Our contribution is not simply implementing three existing models. We establish a controlled experimental comparison of different detector architectures on an Indian toll-plaza-relevant vehicle dataset, analyze class-specific failures, examine the accuracy-latency trade-off, and identify the limitations caused by minority-class representation. This provides an empirical basis for selecting and improving a detector for the next phase."

That is much stronger than:

"We made a vehicle detector."

76. What does Phase 1 actually prove?

Your Phase 1 does **not** prove:

"RT-DETR is universally superior."

It demonstrates:

Under the selected dataset, training setup and model configurations, RT-DETR-I produced the highest observed test-set detection accuracy, while YOLOv8s/YOLOv10s provided substantially lower latency.

That's scientifically defensible.

77. Your strongest result

You can emphasize:

$$RT - DETR = 0.6786$$

versus:

$$YOLOv8 = 0.5847$$

Difference:

$$0.6786 - 0.5847 = 0.0939$$

So RT-DETR improved mAP@50 by about:

$$\frac{0.0939}{0.5847} \times 100 \approx 16.1\%$$

relative to YOLOv8s.

Compared with YOLOv10:

$$0.6786 - 0.5508 = 0.1278$$

or approximately:

$$23.2\%$$

relative improvement.

These are useful numbers to mention.

78. But RT-DETR is slower

RT-DETR:

40.3 ms

YOLOv8:

12.4 ms

So RT-DETR takes approximately:

$$40.3/12.4 \approx 3.25$$

times the latency of YOLOv8 under the reported measurements.

This gives you a clear trade-off:

```
RT-DETR
+ Higher accuracy
- Higher latency
```

YOLOv8

+ Lower latency

- Lower accuracy

79. What would you improve if given more time?

Say:

"The first priority would be addressing the severe class imbalance. I would increase minority-class representation, apply controlled augmentation and investigate class-aware sampling or suitable loss functions. Then I would perform systematic hyperparameter tuning and evaluate robustness under occlusion and small-object conditions. I would also conduct a fairer architecture comparison using models with more comparable parameter sizes."

That last sentence is particularly good.

80. Your biggest technical limitation

Your current results have at least three important limitations:

1. Severe class imbalance

Car >> Bus/Truck

2. Model size isn't controlled

YOLOv8s $\approx$ 11M

YOLOv10s $\approx$ 8M

RT-DETR-l $\approx$ 33M

3. Limited hyperparameter exploration

You trained the models under a particular configuration.

Therefore you can't claim the observed ranking is universally optimal.

81. How Phase 2 can become your research contribution

A strong Phase 2 experiment would be:

```
Phase 1
Baseline comparison
    ↓
RT-DETR best accuracy
    ↓
Identify weakness
    ↓
Class imbalance
    ↓
Phase 2
Improved training strategy
    ↓
Compare
    ↓
Minority-class mAP
    ↓
Overall mAP
    ↓
Latency
    ↓
Final model
```

You could then answer:

"We identified a limitation in Phase 1 and designed Phase 2 specifically to address it."

That gives your project a proper research progression.

82. One thing I would change in your current presentation

Your slide says:

"Phase 1 Conclusion: RT-DETR Superiority"

I'd change it to:

Phase 1 Conclusion: RT-DETR Achieves the Best Accuracy Under Our Experimental Setup

Then:

RT-DETR-I achieved the highest mAP@50 (0.6786), while YOLOv8s and YOLOv10s provided substantially lower latency. The results support our hypothesis that transformer-based contextual reasoning can benefit visually challenging vehicle detection, while highlighting accuracy–latency and class-imbalance trade-offs.

This wording is much more defensible in a viva.

83. The 10 things you MUST memorize

If your evaluator gives you only 5 minutes, know these:

1.

Problem: Vehicle detection/classification in toll-plaza environments.

2.

Dataset: Indian Vehicle Dataset from Kaggle.

3.

Models:

```
YOLOv8s → CNN baseline  
YOLOv10s → NMS-free CNN  
RT-DETR-l → Transformer detector
```

4.

YOLO principle:

Single-stage object detection.

5.

YOLOv10 innovation:

End-to-end NMS-free detection through one-to-one assignment.

6.

RT-DETR innovation:

Transformer-based attention for contextual object detection.

7.

Best accuracy:

```
RT-DETR  
mAP@50 = 0.6786
```

8.

Fastest:

YOLOv10

12.3 ms

9.

Biggest problem:

Severe class imbalance

especially Bus/Truck

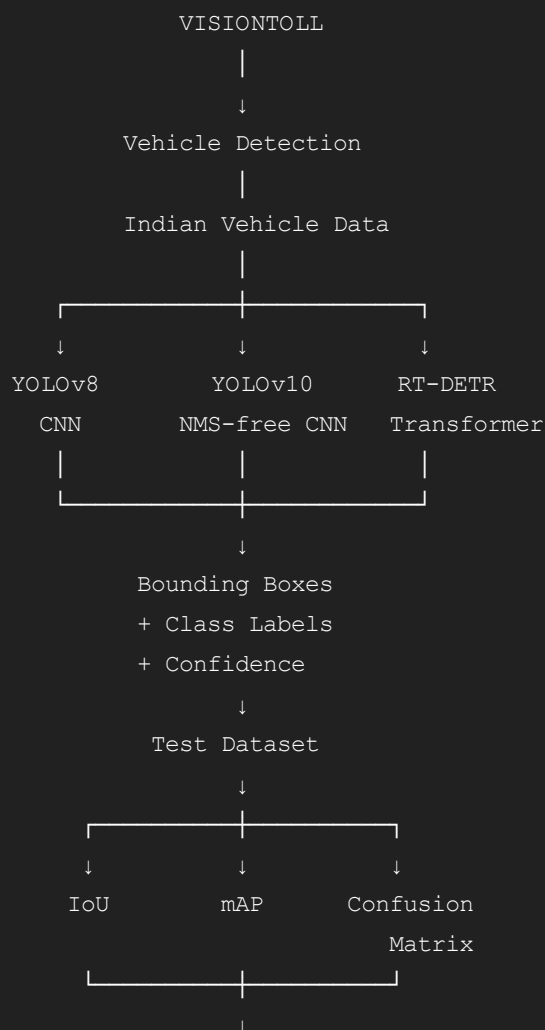
10.

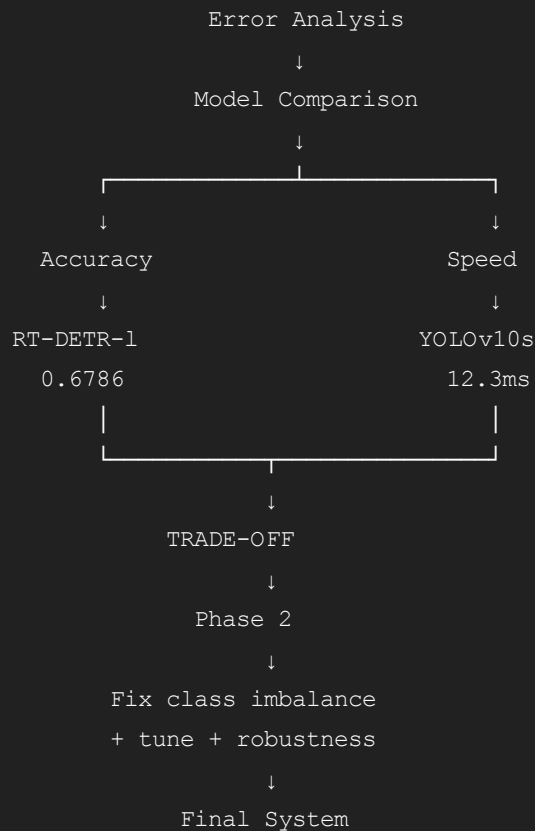
Phase 2:

Improve minority-class representation, augmentation/sampling, tuning, robustness, and final deployment evaluation.

84. Final "if they ask anything" mental model

Keep this picture in your head:





The single most important thing: don't present this as "we trained three models and RT-DETR won." Present it as a **controlled experiment designed to investigate an architectural research question**, with measurable trade-offs and a clear Phase 2 experiment. That framing is what makes the project sound like a proper deep-learning research project rather than just a model implementation.

how many classes can our project recognize?

Your current VisionToll project recognizes **5 vehicle classes**:

1. 🚗 **Car**
2. 🚌 **Bus**
3. 🚚 **Pickup**
4. 🏍️ **Motorcycle**
5. 🚛 **Truck**

So:

Number of classes = 5

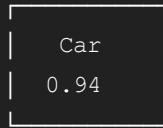
The model doesn't just say "vehicle." It detects the **location (bounding box)** and identifies which of these **5 classes** the vehicle belongs to.

For example:

Input image



VisionToll



Viva answer

If they ask "How many classes does your model recognize?", say:

"Our current dataset contains five vehicle classes: Car, Bus, Pickup, Motorcycle, and Truck. All three models—YOLOv8s, YOLOv10s, and RT-DETR—are trained to detect these same five classes."

One thing to **verify before your evaluation** is that your actual `data.yaml` has exactly these five class names and that the class IDs match your labels. If you show me your `data.yaml`, I can check it for you.

explain me about our results and comapre results

Yes. For your evaluation, you should understand **what each result means, why the models differ, and what conclusion you can legitimately draw.**

1. Your Results at a Glance

Your three models produced:

Model	Architecture	mAP@50	mAP@50-95	Parameters	Latency
YOLOv8s	CNN	0.5847	0.3891	11.1M	12.4 ms
YOLOv10s	NMS-free CNN	0.5508	0.3344	8.1M	12.3 ms
RT-DETR-L	Transformer-based	0.6786	0.4535	32.8M	40.3 ms

The easiest way to remember it:

RT-DETR = best accuracy
YOLOv10 = fastest
YOLOv8 = middle ground

2. First Understand mAP@50

Your most important metric is:

mAP@50

It measures detection accuracy when the predicted bounding box has an IoU threshold of 0.50.

Your results:

RT-DETR	0.6786	
YOLOv8s	0.5847	
YOLOv10s	0.5508	

Therefore:

RT-DETR > YOLOv8s > YOLOv10s

in your experiment.

3. How much better is RT-DETR?

Compared with YOLOv8s:

$$0.6786 - 0.5847 = 0.0939$$

So RT-DETR is **9.39 percentage points higher** in mAP@50.

In relative terms:

$$\frac{0.6786 - 0.5847}{0.5847} \times 100 \approx 16.1\%$$

So you can say:

RT-DETR achieved approximately 16.1% higher mAP@50 than YOLOv8s relative to the YOLOv8s baseline.

Compared with YOLOv10s:

$$0.6786 - 0.5508 = 0.1278$$

That's approximately **23.2% higher relative to YOLOv10s**.

Viva answer

"RT-DETR achieved the highest mAP@50 of 0.6786, which is about 16.1% higher relative to YOLOv8s and 23.2% higher relative to YOLOv10s."

4. What does this actually mean?

It means that **under your experimental setup**, RT-DETR was better at detecting and localizing the vehicle classes.

Remember, your classes are:

- Car
- Bus
- Pickup
- Motorcycle
- Truck

So RT-DETR was the strongest overall detector on your test set.

But it doesn't mean:

"RT-DETR is universally better than YOLO."

It means:

RT-DETR performed better on our dataset and configuration.

That's an important distinction for your viva.

5. Now Look at mAP@50-95

Your results:

RT-DETR	0.4535
YOLOv8s	0.3891
YOLOv10s	0.3344

Again:

RT-DETR > YOLOv8s > YOLOv10s

But why are these numbers lower than $\text{mAP}@50$?

Because $\text{mAP}@50-95$ is much stricter.

It evaluates at multiple IoU thresholds:

```
0.50
0.55
0.60
0.65
...
0.95
```

So the model must produce increasingly accurate bounding boxes.

Therefore:

$\text{mAP}@50$ tells us about detection performance under a relatively forgiving localization threshold, while $\text{mAP}@50-95$ gives a more stringent picture of both detection and localization quality.

6. What does the $\text{mAP}@50-95$ result tell us?

RT-DETR:

0.4535

YOLOv8:

0.3891

YOLOv10:

0.3344

So RT-DETR isn't only finding more correct vehicles—it also produces better localization across stricter IoU thresholds in your experiment.

That's an important point.

7. Now Compare Speed

Your latency:

Model	Latency
YOLOv10s	12.3 ms
YOLOv8s	12.4 ms
RT-DETR-L	40.3 ms

Here:

YOLOv10 > YOLOv8 > RT-DETR

where ">" means faster.

8. Convert latency to FPS

Approximate FPS:

$$FPS = \frac{1000}{Latency}$$

YOLOv8s

$$1000/12.4 \approx 80.6$$

≈ 81 FPS

YOLOv10s

$$1000/12.3 \approx 81.3$$

≈ 81 FPS

RT-DETR

$$1000/40.3 \approx 24.8$$

≈ 25 FPS

So:

Model	Approx FPS
YOLOv10s	81 FPS

Model	Approx FPS
YOLOv8s	81 FPS
RT-DETR-I	25 FPS

These are based on your reported latency measurements; actual end-to-end application FPS can be lower after camera capture, preprocessing, postprocessing, and other overhead.

9. So why is RT-DETR slower?

Look at the model sizes:

```
YOLOv10s    8.1M parameters
YOLOv8s     11.1M parameters
RT-DETR-l   32.8M parameters
```

RT-DETR-I is considerably larger.

It also uses transformer-based processing.

Therefore it has greater computational requirements.

So your result makes sense:

```
More computation
  ↓
Higher latency
  ↓
Lower FPS
```

But you get:

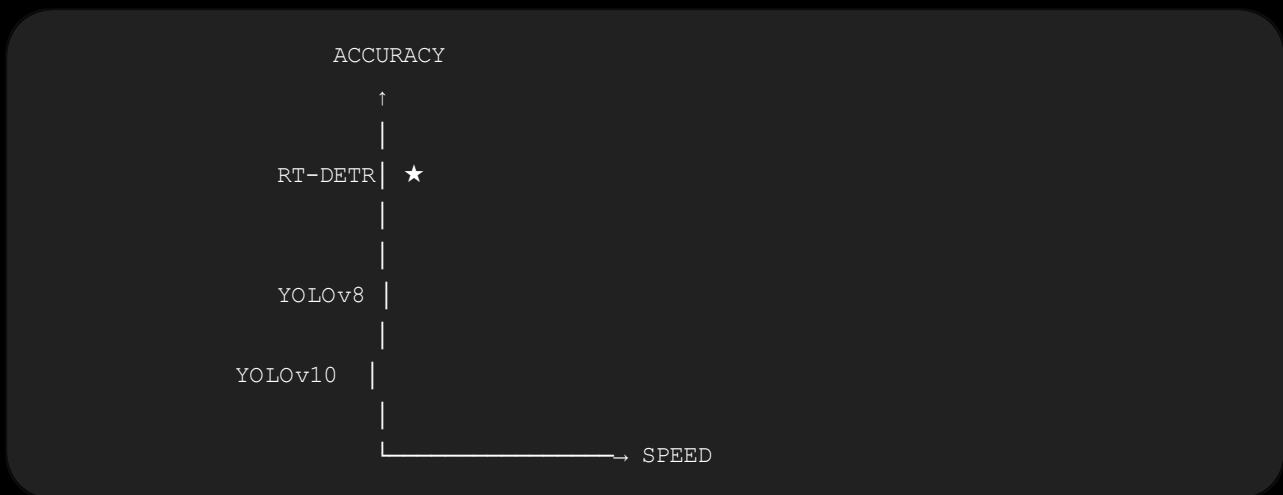
```
Higher model capacity
+
Transformer-based contextual reasoning
  ↓
Higher observed accuracy
```

10. This gives you the most important conclusion

Your results demonstrate an:

Accuracy vs Speed Trade-off

Think of it like this:



RT-DETR

Advantages:

- Highest mAP@50
- Highest mAP@50-95
- Better overall detection performance

Disadvantage:

- Much higher latency

YOLOv8s

Advantages:

- Good accuracy
- Very fast
- Strong baseline

Disadvantage:

- Lower accuracy than RT-DETR

YOLOv10s

Advantages:

- Fastest in your measurements
- Smallest model
- NMS-free design

Disadvantage:

- Lowest accuracy among your three models

11. Why did YOLOv8 beat YOLOv10?

Your results:

```
YOLOv8s
mAP@50 = 0.5847

YOLOv10s
mAP@50 = 0.5508
```

Difference:

$$0.5847 - 0.5508 = 0.0339$$

YOLOv8 is approximately **6.0% higher relative to YOLOv10**.

But don't say:

"YOLOv10 is a worse model."

Instead:

"YOLOv10 performed worse on our particular dataset and configuration. Its NMS-free design gives it a small latency advantage, but that efficiency did not translate into higher accuracy in our experiment."

That's a much better research answer.

12. Why might YOLOv10 struggle?

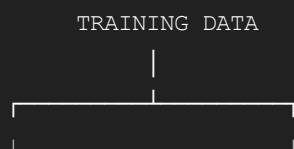
Your dataset has a major problem:

Class imbalance

You reported approximately:

Car	398 instances
Bus	6
Truck	8

So the model sees:



This makes learning minority classes much harder.

And we can see this clearly in your class results.

13. RT-DETR Class-wise Results

Your RT-DETR results:

Class	mAP@50
Car	0.850
Pickup	0.703
Motorcycle	0.680
Truck	0.665
Bus	0.495

Look at the difference:

```
Car      → 0.850  ★
Pickup   → 0.703
Motorcycle → 0.680
Truck     → 0.665
Bus       → 0.495  ⚠
```

This tells us something very important:

The overall 0.6786 mAP is hiding significant class-level differences.

14. Why is Car so good?

You have many more car examples.

Therefore the model gets much more opportunity to learn:

- Shape
- Wheels
- Windows

- Body structure
- Different appearances
- Different orientations

So:

More representative data → easier learning

This doesn't automatically mean the model is overfitting.

It indicates **class imbalance**.

15. Why is Bus difficult?

Your Bus mAP is:

0.495

That's considerably lower than Car.

Possible reasons include:

Very few examples

You reported only around **6 bus instances**.

That's extremely small.

Visual similarity

A bus can look similar to a truck from certain angles.

Perspective

Toll cameras may observe vehicles from elevated or unusual viewpoints.

Occlusion

Another vehicle can hide part of the bus.

Therefore:

```
Few examples
+
Visual similarity
+
Perspective
+
```

Occlusion



Harder classification

16. YOLOv10 class-wise results

Your YOLOv10 results:

Class	mAP@50
Car	0.837
Pickup	0.734
Motorcycle	0.628
Truck	0.344
Bus	0.211

The most striking result:

Car → 0.837
Bus → 0.211
Truck → 0.344

This indicates that YOLOv10 struggled particularly with the heavy-vehicle categories.

17. Compare RT-DETR and YOLOv10 class-wise

This is actually very interesting for your project.

Class	RT-DETR	YOLOv10	Difference
Car	0.850	0.837	+0.013
Bus	0.495	0.211	+0.284
Pickup	0.703	0.734	-0.031
Motorcycle	0.680	0.628	+0.052
Truck	0.665	0.344	+0.321

The biggest improvements are:

Truck

$$0.665 - 0.344 = 0.321$$

Bus

$$0.495 - 0.211 = 0.284$$

That's significant.

And those are precisely the difficult classes.

18. Why is this important for your research gap?

Your research gap involves:

Visually similar vehicle categories.

And what did you observe?

```
RT-DETR
  ↓
Bus = 0.495
Truck = 0.665

YOLOv10
  ↓
Bus = 0.211
Truck = 0.344
```

RT-DETR performs substantially better on those difficult heavy-vehicle classes in your reported results.

This is exactly the kind of evidence you should discuss.

But again, don't claim:

"This proves transformers always solve visual similarity."

Say:

"Our results support the hypothesis that transformer-based contextual representations can be beneficial for visually similar vehicle categories."

19. Why might attention help?

Imagine this truck:



And a bus:



There are shared features:

```
Large rectangular body
+
wheels
+
similar proportions
```

The model needs to understand relationships among different regions.

Transformer attention can help create representations that incorporate broader contextual relationships.

That is the reasoning behind your RT-DETR hypothesis.

20. But there's an important scientific limitation

Your models are not equal in size.

You used:

```
YOLOv8s    → 11.1M
YOLOv10s   → 8.1M
RT-DETR-l  → 32.8M
```

So RT-DETR has substantially more parameters.

Therefore, if the examiner asks:

"How do you know RT-DETR is better because of the transformer rather than because it is a larger model?"

Your answer should be:

"We cannot completely isolate those factors in the current Phase 1 experiment because RT-DETR-L has substantially more parameters. Our Phase 1 conclusion is therefore that RT-DETR-L performed best under our selected configuration, not that transformer architecture alone caused the improvement. A fairer size-matched comparison would be an important Phase 2 experiment."

This is a very good answer.

Don't be afraid to admit limitations. Evaluators usually appreciate scientific honesty.

21. What does your confusion matrix tell you?

The confusion matrix helps answer:

What is the model confusing?

For example:

```
Actual Truck
      ↓
Prediction:
Truck → Correct
Bus   → Incorrect
Car   → Incorrect
```

If you observe significant:

```
Bus ↔ Truck
```

confusion, it supports your claim that heavy vehicles are visually difficult to distinguish.

22. What does your overall comparison tell us?

Let's put everything together.

Accuracy

RT-DETR	0.6786	
YOLOv8s	0.5847	
YOLOv10s	0.5508	

Localization under stricter IoU

RT-DETR	0.4535	
YOLOv8s	0.3891	
YOLOv10s	0.3344	

Speed

YOLOv10s	12.3 ms	
YOLOv8s	12.4 ms	
RT-DETR	40.3 ms	

Model size

YOLOv10s	8.1M	
YOLOv8s	11.1M	
RT-DETR	32.8M	

23. Final ranking

If your priority is accuracy:

 RT-DETR

 YOLOv8s

 YOLOv10s

If your priority is speed:

 YOLOv10s

 YOLOv8s

 RT-DETR

If you want a balanced baseline:

YOLOv8s is a strong middle-ground reference.

24. So what should you call your "winner"?

For your current research objective:

RT-DETR-I is the accuracy winner.

But don't say:

"RT-DETR is the best model."

Say:

"RT-DETR-I is the best-performing model in terms of detection accuracy under our Phase 1 experimental configuration, while YOLOv10s provides the lowest measured latency."

That's your safest and strongest conclusion.

25. Your Phase 1 conclusion

You can explain it like this in your evaluation:

"The experimental results show a clear accuracy-latency trade-off. RT-DETR-I achieved the highest mAP@50 of 0.6786 and mAP@50-95 of 0.4535, outperforming YOLOv8s and YOLOv10s in overall detection accuracy. It also showed substantially better performance on the difficult Bus and Truck classes compared with YOLOv10s. However, RT-DETR-I required 40.3 ms per image, compared with approximately 12 ms for the YOLO models. YOLOv10s was the fastest but had the lowest overall mAP. Therefore, RT-DETR is our Phase 1 accuracy leader, while YOLOv10 represents the low-latency alternative."

26. And your Phase 2 justification

This naturally leads to:

```
PHASE 1
  ↓
RT-DETR best overall
  ↓
BUT...
  ↓
Bus/Truck still weak
  +
Severe class imbalance
```

```
+
RT-DETR larger/slower
↓
PHASE 2
↓
More minority-class data
↓
Better augmentation
↓
Class-aware sampling
↓
Hyperparameter tuning
↓
Fairer model-size comparison
↓
Robustness testing
↓
Final model
```

So your results aren't just numbers.

They tell a **story**:

"RT-DETR gives us the best accuracy, but our analysis reveals that minority classes remain difficult and the model has a speed/size cost. Therefore, Phase 2 will focus on improving minority-class learning and optimizing the final system."

That is the story you should tell during your evaluation.

YOLOv8s → 11.1M
YOLOv10s → 8.1M
RT-DETR-L → 32.8M
what are parameters?
i didnt get this?

Yes — this is **very important to understand for your viva**. Let's make it extremely simple.

What are "parameters"?

A **parameter** is a number that the neural network learns during training.

Think of your model as having millions of tiny adjustable knobs.

Neural Network



↓	↓	↓
Weight 1	Weight 2	Weight 3
↑	↑	↑
adjusted	adjusted	adjusted
during	during	during
training	training	training

These learned numbers are mainly **weights and biases**.

The model changes these numbers during training so that its predictions become better.

Simple example

Imagine we want a model to determine whether something is a **car or truck**.

The model might learn that these features are useful:

```
Wheel shape
Vehicle height
Vehicle length
Window arrangement
Body shape
```

But the model doesn't manually store rules like:

```
"If height > X, call it a truck."
```

Instead, the neural network learns numerical weights.

For example, **conceptually**:

```
Wheel feature      → weight = 0.73
Height feature     → weight = 0.91
Length feature     → weight = 0.62
Window feature     → weight = 0.35
```

These numbers are learned automatically.

Real neural networks have millions of such numbers.

So what does 11.1M mean?

When we say:

```
YOLOv8s = 11.1M parameters
```

it means approximately:

11.1 million

learnable numerical values exist in the model.

Similarly:

```
YOLOv8s      → 11.1 million  
YOLOv10s     →  8.1 million  
RT-DETR-l    → 32.8 million
```

So RT-DETR-l has roughly **32.8 million learned parameters**.

Think of it like a student's brain

This is only an analogy, but useful for understanding.

Imagine three students:

```
Student A  
YOLOv10s  
8.1M "adjustable knowledge values"  
  
Student B  
YOLOv8s  
11.1M "adjustable knowledge values"  
  
Student C  
RT-DETR-l  
32.8M "adjustable knowledge values"
```

The number of parameters is somewhat like the **capacity of the model to represent patterns**.

More parameters can give a model more capacity, but:

More parameters does NOT automatically mean better performance.

That's extremely important.

How are parameters learned?

Suppose you give YOLOv8 an image:

```
IMAGE
↓
YOLOv8 initially
makes prediction
↓
"This is a bus"
↓
Ground truth says
"This is truck"
↓
LOSS
↓
Backpropagation
↓
Update parameters
↓
Try again on next
examples
```

The parameters are continuously adjusted during training.

After 50 epochs, the model has learned a set of parameter values that hopefully allows it to detect vehicles accurately.

What does a parameter actually look like?

Inside a neural network, you might have something like:

```
Layer
↓
[ 0.12  -0.43   0.87   0.21
  -0.55   0.31  -0.09   0.76
   0.44   0.18  -0.62   0.91 ]
```

Those are just example numbers.

There can be **millions** of them distributed across the model's layers.

During training:

```
Before training:

0.12 → -0.43 → 0.87 → ...

↓
TRAINING
```

↓

After training:

0.18 → -0.51 → 0.92 → ...

The network has learned better values.

Parameters vs layers

This is another thing students commonly confuse.

Layer

A **structural component** of the neural network.

For example:

```
Input
↓
Convolution layer
↓
Convolution layer
↓
Feature layer
↓
Detection head
```

Parameter

A **learnable number** inside those layers.

So:

```
Layers
↓
contain weights/biases
↓
which are parameters
```

Why does RT-DETR have 32.8M?

Your RT-DETR model is:

RT-DETR-l

The l means a larger model variant.

It has substantially more parameters:

YOLOv10s	8.1M
YOLOv8s	11.1M
RT-DETR-l	32.8M

That means RT-DETR-l has much greater model capacity.

It also means it generally requires more computation.

That's one reason your results show:

	PARAMETERS	LATENCY
YOLOv10s	8.1M	12.3 ms
YOLOv8s	11.1M	12.4 ms
RT-DETR-l	32.8M	40.3 ms

Notice the relationship:

```
More parameters
  ↓
More computation
  ↓
Potentially higher latency
```

It's not a strict one-to-one relationship, though. Architecture, operations, hardware, input size, and implementation also affect latency.

Does more parameters mean more accuracy?

Not necessarily.

For example, imagine:

Model A
10M parameters
mAP = 0.60
Model B

Model B has 3× the parameters but performs worse.

Why?

Because performance depends on:

- Architecture
- Training data
- Data quality
- Hyperparameters
- Optimization
- Model capacity
- Generalization
- Class distribution

So don't say:

"RT-DETR is better because it has more parameters."

Instead say:

"RT-DETR has substantially more parameters, which gives it greater model capacity, but its higher accuracy cannot be attributed to parameter count alone because architecture and training also influence performance."

VERY important for YOUR project

This is actually a limitation in your current comparison.

You're comparing:

```
YOLOv8s
11.1M parameters

YOLOv10s
8.1M parameters

RT-DETR-l
32.8M parameters
```

RT-DETR-l is much larger.

Therefore, your experiment isn't a perfectly **parameter-matched comparison**.

If your evaluator asks:

"Why is RT-DETR better? Is it because it is a Transformer or because it has more parameters?"

You should answer:

"Our Phase 1 results show that RT-DETR-l performs better under our selected configuration, but we cannot isolate the effect of the Transformer architecture from the effect of its larger model capacity. A parameter- or model-size-matched comparison would be a useful Phase 2 experiment."

That's a **very strong research answer**.

One final analogy

Think of parameters as **adjustable knobs** on a machine.

```
YOLOv10s
8.1 million knobs
    ↓
YOLOv8s
11.1 million knobs
    ↓
RT-DETR-l
32.8 million knobs
```

During training, the neural network learns:

"Which setting should every knob have so that I can correctly detect cars, buses, pickups, motorcycles and trucks?"

After training, those millions of values collectively encode what the model has learned about visual patterns.

For your viva, remember this sentence:

"Parameters are the learnable numerical values, mainly weights and biases, that are adjusted during training. Our YOLOv8s has about 11.1 million, YOLOv10s about 8.1 million, and RT-DETR-l about 32.8 million parameters."

I can also explain **exactly what a weight, bias, neuron, convolution, and feature map are**, which will make the architecture diagrams much easier to understand for your viva.

[See another version](#)

Do you like this personality?

ChatGPT can make mistakes. Check important info.